

[chromium](#) / [chromiumos](#) / [platform](#) / [installer](#) / [master](#) / [.](#) / **chromeos-install**blob: 1d00a79e3c7160a2869fd9b7fc1a2e6004a4c191 [[file](#)]

```
1  #!/bin/sh -u
2  # Copyright (c) 2012 The Chromium OS Authors. All rights reserved.
3  # Use of this source code is governed by a BSD-style license that can be
4  # found in the LICENSE file.
5  #
6  # A script to install from removable media to hard disk.
7
8  # If we're not running as root, restart as root.
9  if [ ${UID:-$(id -u)} -ne 0 ]; then
10     exec sudo "$0" "$@"
11 fi
12
13 # Load functions and constants for chromeos-install.
14 . /usr/share/misc/chromeos-common.sh || exit 1
15 . /usr/share/misc/shflags || exit 1
16
17 # TODO(installer): Clean up all these flags. There are way too many flags in
18 # this script.
19
20 DEFINE_string dst "" "Destination device"
21 DEFINE_boolean skip_src_removable ${FLAGS_FALSE} \
22     "Skip check to ensure source is removable"
23 DEFINE_boolean skip_dst_removable ${FLAGS_FALSE} \
24     "Skip check to ensure destination is not removable"
25 DEFINE_boolean skip_rootfs ${FLAGS_FALSE} \
26     "Skip installing the rootfs; Only set up partition table"
27 DEFINE_boolean yes ${FLAGS_FALSE} \
28     "Answer yes to everything"
29 DEFINE_boolean skip_vblock ${FLAGS_FALSE} \
30     "Skip copying the HD vblock to install destination"
31 DEFINE_boolean preserve_stateful ${FLAGS_FALSE} \
32     "Don't create a new filesystem for the stateful partition. Be careful \
33 using this option as this may make the stateful partition not mountable."
34 DEFINE_string arch "" \
35     "Architecture for this image, must be one of 'ARM' or 'INTEL'. If \
36 unset, auto-detect."
37 DEFINE_string payload_image "" "Path to a Chromium OS image to install onto \
38 the device's hard drive"
```

```

39  DEFINE_boolean use_payload_kern_b ${FLAGS_FALSE} \
40      "Copy KERN-B instead of KERN-A from payload_image."
41  DEFINE_string gpt_layout "" "Path to a script for pre-defined GPT partition \
42  layout"
43  DEFINE_string pmbr_code "" "Path to PMBR code to be installed"
44  DEFINE_string target_bios "" "Bios type to boot with (see postinst --bios)"
45  DEFINE_boolean mtd_layout ${FLAGS_FALSE} "This system uses MTD for \
46  partitioning rather than GPT"
47  DEFINE_boolean debug ${FLAGS_FALSE} "Show debug output"
48  DEFINE_boolean large_test_partitions ${FLAGS_FALSE} \
49      "Make partitions 9 and 10 large (for filesystem testing)"
50  DEFINE_boolean skip_postinstall ${FLAGS_FALSE} \
51      "Skip postinstall for situations where you're building for a \
52  non-native arch. Note that this will probably break verity."
53
54
55  # Parse command line
56  FLAGS "$@" || exit 1
57  eval set -- "${FLAGS_ARGV}"
58
59  # Be aggressive.
60  set -eu
61
62  SUB_CMD_DEBUG_FLAG=""
63  if [ "$FLAGS_debug" -eq "${FLAGS_TRUE}" ]; then
64      set -x
65      SUB_CMD_DEBUG_FLAG="--debug"
66  fi
67
68  # Determine our architecture
69  if [ -z "$FLAGS_arch" ]; then
70      case $(uname -m) in
71          i?86|x86_64)
72              ARCH="INTEL"
73              ;;
74          arm*)
75              ARCH="ARM"
76              ;;
77          *)
78              echo "Error: Failed to auto detect architecture" >&2
79              exit 1
80      esac
81  else
82      case ${FLAGS_arch} in
83          INTEL|ARM)
84              ;;

```

```

85     *)
86     echo "Error: Unknown architecture '$FLAGS_arch'." >& 2
87     exit 1
88     esac
89     ARCH="$FLAGS_arch"
90 fi
91
92 fast_dd() {
93     # Usage: fast_dd <block size> <count> <seek> <skip> other dd args
94     local user_block_size="$1"
95     shift
96     local user_count="$1"
97     shift
98     local user_seek="$1"
99     shift
100    local user_skip="$1"
101    shift
102    local ideal_block_size=$((2 * 1024 * 1024)) # 2 MiB
103    if [ $((($ideal_block_size % $user_block_size)) -eq 0) ]; then
104        local factor=$((($ideal_block_size / $user_block_size))
105        if [ $((($user_count % $factor)) -eq 0 -a \
106            $((($user_seek % $factor)) -eq 0 -a \
107            $((($user_skip % $factor)) -eq 0) ]; then
108            local count_arg=""
109            if [ "$user_count" -ne 0 ]; then
110                count_arg="count=$((($user_count / $factor))"
111            fi
112            dd $* bs="$ideal_block_size" seek=$((($user_seek / $factor)) \
113                skip=$((($user_skip / $factor)) $count_arg
114            return
115        fi
116    fi
117    # Give up and do the user's slow dd
118    echo
119    echo WARNING: DOING A SLOW dd OPERATION. PLEASE FIX
120    echo
121    local count_arg=""
122    if [ "$user_count" -ne 0 ]; then
123        count_arg="count=$user_count"
124    fi
125    dd $* bs="$user_block_size" seek="$user_seek" skip="$user_skip" \
126        $count_arg
127 }
128
129 get_format() {
130     local part=$1

```

```

131     eval echo \${FORMAT_${part}}
132 }
133
134 get_fs_format() {
135     local part=$1
136     eval echo \${FS_FORMAT_${part}}
137 }
138
139 # Update a specific partition in the destination device.
140 write_partition() {
141     local user_part="$1"
142     local user_count="$2"
143     local user_seek="$3"
144     local user_skip="$4"
145     local src="$5"
146     local dst="$6"
147     local format fs_format
148
149     if [ ${user_count} -eq 0 ]; then
150         echo "Skipping partition as it does not exist"
151         return 0
152     fi
153
154     format=$(get_format ${user_part})
155     case ${format} in
156     nand)
157         flash_erase "/dev/mtd${user_part}" 0 0
158         fast_dd 512 "${user_count}" "0" "${user_skip}" \
159             if="${src}" of="/dev/mtdblock${user_part}" conv=notrunc
160         ;;
161     ubi)
162         local phy_ubi="/dev/ubi${user_part}"
163         local log_ubi="${phy_ubi}_0"
164         local sysfs_name="/sys/class/mtd/mtd${user_part}/name"
165
166         # Do we have to initialize it first?
167         if [ ! -e ${phy_ubi} ]; then
168             ubiformat -y -e 0 /dev/mtd${user_part}
169             ubiattach -d ${user_part} -m ${user_part}
170         fi
171         if [ ! -e ${log_ubi} ]; then
172             ubimkvol -m ${phy_ubi} -N "$(cat "${sysfs_name}")"
173         fi
174
175         fs_format=$(get_fs_format ${user_part})

```

```

177     case ${fs_format} in
178     ubifs)
179         local src_mnt="${TMPMNT}/src" dst_mnt="${TMPMNT}/dst"
180         mkdir -p "${src_mnt}" "${dst_mnt}"
181
182         mkfs.ubifs -y -x none -R 0 "${log_ubi}"
183         mount "${log_ubi}" "${dst_mnt}"
184
185         # Have to copy the files over by hand as the source partition is a
186         # different filesystem type (like ext4).
187         loop_offset_setup "${src}" "${user_seek}"
188         TMPMNT="${src_mnt}" mount_on_loop_dev
189         cp -a "${src_mnt}"/* "${dst_mnt}"/
190         TMPMNT="${src_mnt}" umount_from_loop_dev
191         loop_offset_cleanup
192
193         umount "${log_ubi}"
194         ;;
195     *)
196         ubiupdatevol --skip $((user_skip * 512)) --size $((user_count * 512)) \
197             ${log_ubi} "${src}"
198         ;;
199     esac
200     ;;
201
202 *)
203     fast_dd 512 "${user_count}" "${user_seek}" "${user_skip}" \
204         if="${src}" of="${dst}" conv=notrunc
205     ;;
206 esac
207 }
208
209 # Find root partition of the block device that we are installing from
210 get_root_device() {
211     rootdev -s
212 }
213
214 # Check for optional payload image
215 if [ "$FLAGS_skip_rootfs" -eq "$FLAGS_TRUE" -a -s "$FLAGS_gpt_layout" ]; then
216     # Usually this is used for partition setup.
217     SRC=""
218     ROOT=""
219 elif [ -z "$FLAGS_payload_image" ]; then
220     # Find root partition of the root block device
221     SRC=$(get_block_dev_from_partition_dev $(get_root_device))
222     ROOT=""

```

```

223
224     if [ "$FLAGS_skip_src_removable" -eq "${FLAGS_FALSE}" ]; then
225         if [ "$(cat /sys/block/${SRC#/dev}/removable)" != "1" ]; then
226             # Work around ARM kernel bug http://crosbug.com/14871
227             # Removable flag is implemented inconsistently for ARM sdcard reader.
228             if [ "${SRC}" != "/dev/mmcblk1" ]; then
229                 echo "Error: Source does not look like a removable device: $SRC"
230                 exit 1
231             fi
232         fi
233     fi
234 else
235     if [ ! -e "$FLAGS_payload_image" ]; then
236         echo "Error: No payload image found at $FLAGS_payload_image"
237         exit 1
238     fi
239
240     # Needed to copy PMBR code off image
241     SRC="$FLAGS_payload_image"
242     ROOT="$(mktemp -d)"
243 fi
244
245 #####
246 # Helpful constants and functions.
247
248 # Where to store any related hardware diagnostics data.
249 HARDWARE_DIAGNOSTICS_PATH=/tmp/hardware_diagnostics.log
250 PMBRCODE=/tmp/gptmbr.bin
251 TMPMNT=/tmp/install-mount-point
252 mkdir -p ${TMPMNT}
253
254 # Clean any mounts that might be present to avoid
255 # aliasing access to block devices.
256 prepare_disk() {
257     if [ -e /etc/init/cros-disks.conf ]; then
258         initctl stop cros-disks || true
259     fi
260     # Often times, nothing is mounted, so swallow the warnings.
261     umount -f /media/*/* 2>&1 | \
262     grep -v -i \
263         -e 'No such file or directory' \
264         -e 'not found' || true
265 }
266
267 # Like mount but keeps track of the current mounts so that they can be cleaned
268 # up automatically.

```

```

269 tracked_mount() {
270     local last_arg
271     eval last_arg=\${$#}
272     MOUNTS="\${last_arg}${MOUNTS:+ }${MOUNTS:-}"
273     mount "$@"
274 }
275
276 # Unmount with tracking.
277 tracked_umount() {
278     # dash doesnt support ${//} expansions
279     local new_mounts
280     for mount in $MOUNTS; do
281         if [ "$mount" != "$1" ]; then
282             new_mounts="\${new_mounts:-}${new_mounts+ }$mount"
283         fi
284     done
285     MOUNTS=${new_mounts:-}
286
287     umount "$1"
288 }
289
290 # Create a loop device on the given file at a specified (sector) offset.
291 # Remember the loop device using the global variable LOOP_DEV.
292 # Invoke as: command
293 # Args: FILE OFFSET
294 loop_offset_setup() {
295     local filename=$1
296     local offset=$2
297
298     LOOP_DEV=$(losetup -f)
299     if [ -z "$LOOP_DEV" ]; then
300         echo "No free loop device. Free up a loop device or reboot. Exiting."
301         exit 1
302     fi
303     losetup -o $((offset * 512)) ${LOOP_DEV} ${filename}
304
305     LOOPS="\${LOOP_DEV}${LOOPS:+ }${LOOPS:-}"
306 }
307
308 # Delete the current loop device.
309 loop_offset_cleanup() {
310     # dash doesnt support ${//} expansions
311     local new_loops
312     for loop in $LOOPS; do
313         if [ "$loop" != "$LOOP_DEV" ]; then
314             new_loops="\${new_loops:-}${new_loops+ }$loop"

```

```

315     fi
316 done
317 LOOPS=${new_loops:-}
318
319     # losetup -a doesn't always show every active device, so we'll always try to
320     # delete what we think is the active one without checking first. Report
321     # success no matter what.
322     losetup -d ${LOOP_DEV} || /bin/true
323 }
324
325 # Mount the existing loop device at the mountpoint in $TMPMNT.
326 # Args: optional 'readwrite'. If present, mount read-write, otherwise read-only.
327 mount_on_loop_dev() {
328     local rw_flag=${1-readonly}
329     local mount_flags=""
330     if [ "${rw_flag}" != "readwrite" ]; then
331         mount_flags="-o ro"
332     fi
333     tracked_mount ${mount_flags} ${LOOP_DEV} ${TMPMNT}
334 }
335
336 # Unmount loop-mounted device.
337 umount_from_loop_dev() {
338     mount | grep -q " on ${TMPMNT} " && tracked_umount ${TMPMNT}
339 }
340
341 # Check if all arguments are non-empty values
342 check_non_empty_values() {
343     local value
344     for value in "$@"; do
345         if [ -z "$value" ]; then
346             return ${FLAGS_FALSE}
347         fi
348     done
349     return ${FLAGS_TRUE}
350 }
351
352 # Undo all mounts and loops and runs hw diagnostics on failure.
353 cleanup_on_failure() {
354     set +e
355
356     # Generate the diagnostics log that can be used by a caller.
357     echo "Running a hw diagnostics test -- this might take a couple minutes."
358     badblocks -sv "${DST}" 2>&1 | tee "${HARDWARE_DIAGNOSTICS_PATH}"
359
360     # Run a few extra diagnostics and then them to stdout. These will be stored

```

```

361     # as part of the recovery.log for recovery images.
362     if which smartctl >/dev/null 2>&1; then
363         smartctl -l error "${DST}"
364         smartctl -l selftest "${DST}"
365         smartctl -a "${DST}"
366     fi
367
368     cleanup
369 }
370
371 # Undo all mounts and loops.
372 cleanup() {
373     set +e
374
375     local mount_point
376     for mount_point in ${MOUNTS:-}; do
377         umount "$mount_point" || /bin/true
378     done
379     MOUNTS=""
380
381     local loop_dev
382     for loop_dev in ${LOOPS:-}; do
383         losetup -d "$loop_dev" || /bin/true
384     done
385     LOOPS=""
386
387     if [ ! -z "$ROOT" ]; then
388         rmdir "$ROOT"
389     fi
390 }
391
392 # Copy kernel verification block to target HD
393 # TODO(installer): Remove once crbug.com/320117 is fixed.
394 copy_kernel_vblock() {
395     local ret=0
396
397     if [ -s $VERIFY_BLOB ]; then
398         if [ ${FLAGS_mtd_layout} -eq ${FLAGS_FALSE} ]; then
399             fast_dd 512 0 ${START_KERN_A} 0 if="${VERIFY_BLOB}" \
400                 of="${DST}" conv=notrunc
401             fast_dd 512 0 ${START_KERN_B} 0 if="${VERIFY_BLOB}" \
402                 of="${DST}" conv=notrunc
403         else
404             # We have to erase before we write else you get a boatload of I/O errors
405             flash_erase "/dev/mtd${KERNEL_PART}" 0 0
406             flash_erase "/dev/mtd${KERNEL_B_PART}" 0 0

```

```

407
408     fast_dd 512 0 ${START_KERN_A} 0 if="${VERIFY_BLOB}" \
409         of="/dev/mtdblock${KERNEL_PART}" conv=notrunc
410     fast_dd 512 0 ${START_KERN_B} 0 if="${VERIFY_BLOB}" \
411         of="/dev/mtdblock${KERNEL_B_PART}" conv=notrunc
412     fi
413
414     ret=1
415 fi
416 echo ${ret}
417 }
418
419 check_removable() {
420     if [ ${FLAGS_skip_dst_removable} -eq ${FLAGS_TRUE} ]; then
421         return
422     fi
423
424     local removable
425
426     if ! removable=$(cat /sys/block/${DST#/dev/}/removable); then
427         echo "Error: Invalid destination device (must be whole device): ${DST}"
428         exit 1
429     fi
430
431     if [ "${removable}" != "0" ]; then
432         echo "Error: Attempt to install to a removeable device: ${DST}"
433         exit 1
434     fi
435 }
436
437 STATEFUL_PART=1
438 # This is defined later once we have mounted the ROOT.
439 STATEFUL_FORMAT=
440
441 # Wipes and expands the stateful partition.
442 wipe_stateful() {
443     echo "Clearing the stateful partition..."
444     local stateful_fs_format=$(get_fs_format ${STATEFUL_PART})
445
446     case ${STATEFUL_FORMAT} in
447     ubi)
448         local phy_ubi="/dev/ubi${STATEFUL_PART}"
449         local log_ubi="${phy_ubi}_0"
450         local sysfs_name="/sys/class/mtd/mtd${STATEFUL_PART}/name"
451
452         # Do we have to initialize it first?

```

```

453     if [ ! -e ${phy_ubi} ]; then
454         ubiformat -y -e 0 /dev/mtd${STATEFUL_PART}
455         ubiattach -d ${STATEFUL_PART} -m ${STATEFUL_PART}
456     fi
457     if [ ! -e ${log_ubi} ]; then
458         ubimkvol -m ${phy_ubi} -N "$(cat "${sysfs_name}")"
459     fi
460     ;;
461 *)
462     loop_offset_setup ${DST} ${START_STATEFUL}
463     ;;
464 esac

465
466 # We always make any ext* stateful partitions ext4.
467 case ${stateful_fs_format} in
468 ext[234])
469     mkfs.ext4 -F -b 4096 -L "H-STATE" ${LOOP_DEV} \
470         $((NUM_STATEFUL_SECTORS / 8))
471     ;;
472 ubifs)
473     mkfs.ubifs -y -x none -R 0 /dev/ubi${STATEFUL_PART}_0
474     ;;
475 esac

476
477 case ${STATEFUL_FORMAT} in
478 ubi) ;;
479 *)
480     # Need to synchronize before releasing loop device, otherwise calling
481     # loop_offset_cleanup may return "device busy" error.
482     sync
483     loop_offset_cleanup
484     ;;
485 esac
486 }

487
488 # Install the stateful partition content
489 # Method handles copying data over to the stateful partition. This is done
490 # differently than other partitions due to the EXPAND option i.e. src partition
491 # and dst partitions are of different sizes. In addition, there are some special
492 # tweaks we do for stateful here for various workflows.
493 install_stateful() {
494     # In general, the system isn't allowed to depend on anything
495     # being in the stateful partition at startup. We make some
496     # exceptions for dev images (only), as enumerated below:
497     #
498     # var_overlay

```

```

499 # These are included to support gmerge, and must be kept in
500 # sync with those listed in /etc/init/var-overlay.conf:
501 #     db/pkg
502 #     lib/portage
503 #
504 # dev_image
505 # This provides tools specifically chosen to be mounted at
506 # /usr/local as development only tools.
507 #
508 # Every exception added makes the dev image different from
509 # the release image, which could mask bugs. Make sure every
510 # item you add here is well justified.
511 echo "Installing the stateful partition..."
512 case ${STATEFUL_FORMAT} in
513     ubi)
514         # We modify the global used here as it affects how we unmount later.
515         LOOP_DEV="/dev/ubi${STATEFUL_PART}_0"
516         ;;
517     *)
518         loop_offset_setup ${DST} ${START_STATEFUL}
519         ;;
520     esac
521 mount_on_loop_dev readwrite
522
523 # Whitelist files to copy onto the stateful partition.
524 #
525 # When adding to the whitelist, consider the need for related changes in
526 # src/platform/init/chromeos_startup, and in src/platform/dev/stateful_update.
527 #
528 local dirlist="unencrypted/import_extensions"
529 if crosssystem 'cros_debug?1'; then
530     dirlist="
531         ${dirlist}
532         var_overlay/db/pkg
533         var_overlay/lib/portage
534         dev_image
535     "
536 fi
537
538 if crosssystem 'devsw_boot?1' ; then
539     # This is a base build, and the dev switch was on when we booted;
540     # we assume it will be on for the next boot. We touch
541     # ".developer_mode" to avoid a pointless delay after reboot while
542     # chromeos_startup wipes an empty stateful partition.
543     #
544     # See chromeos_startup for the companion code that checks for this

```

```

545     # file.
546     #
547     touch ${TMPMNT}/.developer_mode
548 fi
549
550     # Does our copy of 'cp' support -u? Not sure why -u is desired, but
551     # may be related to --preserve_stateful.
552     #
553     local opt_u=""
554     if cp -u /dev/null /dev/zero 2>/dev/null; then
555         opt_u="u"
556     fi
557
558     local dir
559     for dir in ${dirlist}; do
560         if [ ! -d "${ROOT}/mnt/stateful_partition/${dir}" ]; then
561             continue
562         fi
563         local parent=$(dirname "${dir}")
564         mkdir -p "${TMPMNT}/${parent}"
565         cp -a${opt_u} "${ROOT}/mnt/stateful_partition/${dir}" "${TMPMNT}/${dir}"
566     done
567
568     umount_from_loop_dev
569     case ${STATEFUL_FORMAT} in
570     ubi) ;;
571     *)
572         loop_offset_cleanup
573         ;;
574     esac
575 }
576
577     # Partition numbers that have assumptions about them. This list should be kept
578     # to a minimal. Check copy_partition for most special casing.
579     KERNEL_A_PART=2
580     ROOTFS_A_PART=3
581     KERNEL_B_PART=4
582     ROOTFS_B_PART=5
583
584     # Copy partition from src to dst (figures out partition offsets). Note, this
585     # has some special casing for rootfs, kernel, and stateful partitions. In
586     # addition, it only copies partitions that are equally sized over one another.
587     # $1 - Partition number we are copying to.
588     # $2 - src image
589     # $3 - dst image.
590     copy_partition() {

```

```

591 local part_num=$1
592 local src=$2
593 local dst=$3
594 local part_size=$(partsize ${src} ${part_num})
595 local src_offset=$(partoffset ${src} ${part_num})
596 local dst_offset=$(partoffset ${dst} ${part_num})
597
598 echo "Installing partition ${part_num} to ${dst}"
599
600 case ${part_num} in
601     ${STATEFUL_PART})
602         install_stateful
603         ;;
604     ${ROOTFS_A_PART}|${ROOTFS_B_PART})
605         # Always copy from ROOT_A for rootfs partitions.
606         part_size=$(partsize ${src} ${ROOTFS_A_PART})
607         src_offset=$(partoffset ${src} ${ROOTFS_A_PART})
608         write_partition ${part_num} ${part_size} ${dst_offset} \
609             ${src_offset} ${src} ${dst}
610         ;;
611     ${KERNEL_A_PART}|${KERNEL_B_PART})
612         if [ ${FLAGS_use_payload_kern_b} -eq ${FLAGS_FALSE} ]; then
613             part_size=$(partsize ${src} ${KERNEL_A_PART})
614             src_offset=$(partoffset ${src} ${KERNEL_A_PART})
615         else
616             part_size=$(partsize ${src} ${KERNEL_B_PART})
617             src_offset=$(partoffset ${src} ${KERNEL_B_PART})
618         fi
619         write_partition ${part_num} ${part_size} ${dst_offset} \
620             ${src_offset} ${src} ${dst}
621         ;;
622     *)
623         local dst_part_size=$(partsize ${dst} ${part_num})
624         if [ ${part_size} -ne ${dst_part_size} -o ${part_size} -le 4096 ]; then
625             # We only copy partitions that are equally sized and greater than the
626             # min fs block size. This matches the build_image logic.
627             return
628         fi
629         write_partition ${part_num} ${part_size} ${dst_offset} \
630             ${src_offset} ${src} ${dst}
631         ;;
632     esac
633 }
634
635 #####
636

```

```

637 # Start main logic. As little code as possible should go in here. Please make
638 # sure put any helper methods above this point to make this main component as
639 # simple as possible.
640
641 # We untrap on success and run cleanup ourselves. Otherwise, on any failure, run
642 # our custom trap method to gather any diagnostic data before cleaning up.
643 trap cleanup_on_failure EXIT
644
645 # Clean media browser mounts if they've popped up.
646 prepare_disk
647
648 locate_gpt
649
650 # Special handling for payload_image. This is passed in for recovery images
651 # and usb installs. This is done first so we can read the gpt partition
652 # file below.
653 if [ -n "${FLAGS_payload_image}" ]; then
654     SRC="${FLAGS_payload_image}"
655     # Mount files that are required to be referenced (when not already mounted).
656     loop_offset_setup "${SRC}" $(partoffset "${SRC}" ${ROOTFS_A_PART})
657     tracked_mount -o ro "${LOOP_DEV}" "${ROOT}"
658     loop_offset_setup "${SRC}" $(partoffset "${SRC}" ${STATEFUL_PART})
659     tracked_mount -o ro "${LOOP_DEV}" "${ROOT}"/mnt/stateful_partition
660 fi
661
662 # Load the GPT helper functions and the image settings.
663 . "${ROOT}"/usr/sbin/write_gpt.sh"
664 load_base_vars
665
666 # Now that we have loaded the partition table we can actually read the format
667 # and partition information.
668 STATEFUL_FORMAT=$(get_format ${STATEFUL_PART})
669
670 # Find our destination device.
671 # If the user hasn't selected a destination,
672 # we expect that the disk layout declares it for us.
673 DST=${FLAGS_dst:-$(get_fixed_dst_drive)}
674 if [ -z "${DST}" ]; then
675     echo "Error: can not determine destination device. Specify --dst yourself."
676     exit 1
677 fi
678
679 if [ "${DST}" = "/dev/ubi" ]; then
680     FLAGS_mtd_layout=${FLAGS_TRUE}
681 fi
682

```

```

683  # Check out the dst device.
684  if [ ${FLAGS_mtd_layout} -eq ${FLAGS_TRUE} ]; then
685      FLAGS_skip_dst_removable=${FLAGS_TRUE}
686  elif [ ! -b "${DST}" ]; then
687      echo "Error: Unable to find destination block device: ${DST}"
688      exit 1
689  fi
690
691  check_removable
692
693  if [ "${DST}" = "${SRC}" ]; then
694      echo "Error: src and dst are the same: $SRC = $DST"
695      exit 1
696  fi
697
698  # Ask for confirmation to be sure.
699  echo "This will install from '$SRC' to '$DST'."
700  echo "This will erase all data at this destination: $DST"
701  if [ ${FLAGS_yes} -eq ${FLAGS_FALSE} ]; then
702      read -p "Are you sure (y/N)? " SURE
703      if [ "$SURE" != "y" ]; then
704          echo "Ok, better safe than sorry; you answered '$SURE'."
705          # Don't run diagnostics if the user explicitly bailed out.
706          trap - EXIT
707          cleanup
708          exit 1
709      fi
710  fi
711
712  if [ -n "${FLAGS_pmbr_code}" ]; then
713      PMBRCODE="${FLAGS_pmbr_code}"
714  elif [ ${FLAGS_mtd_layout} -eq ${FLAGS_TRUE} ]; then
715      : nop it
716  elif [ "$ARCH" = "ARM" ]; then
717      # Store the existing PMBR from the destination in the temp pmbr file so we can
718      # restore it when we're installing the GPT. This ensures that we preserve
719      # settings set by crossystem (such as dev_boot_usb).
720      dd bs=512 count=1 if=$DST of=$PMBRCODE
721  else
722      # Steal the PMBR code from the source MBR to put on the dest MBR, for booting
723      # on legacy-BIOS devices.
724      dd bs=512 count=1 if=$SRC of=$PMBRCODE
725  fi
726
727  # Write the GPT using the board specific script.
728  if [ ${FLAGS_mtd_layout} -eq ${FLAGS_FALSE} ]; then

```

```

729     write_base_table ${DST} ${PMBRCODE}
730     /sbin/blockdev --rereadpt ${DST}
731     legacy_offset_size_export "${DST}"
732 else
733     legacy_offset_size_export "${SRC}"
734 fi
735
736 if [ ${FLAGS_skip_rootfs} -eq ${FLAGS_TRUE} ]; then
737     echo Done installing partitons.
738     exit 0
739 fi
740
741 if [ ${FLAGS_preserve_stateful} -eq ${FLAGS_FALSE} ]; then
742     wipe_stateful
743 fi
744
745 # Install the content. Do so in reverse order to have stateful get installed
746 # last. The order shouldn't matter but legacy behavior has us go in reverse
747 # order.
748 for CURRENT_PART in $(seq 12 -1 1); do
749     copy_partition ${CURRENT_PART} ${SRC} ${DST}
750 done
751
752
753 #####
754 # Post partition copying work and special casing
755 #####
756
757 # The recovery image is signed with different keys from the hard-disk image. We
758 # need to copy the hard disk verification block from the stateful partition so
759 # that reboot from HD can be verified.
760 VERIFY_BLOB="${RROOT}/mnt/stateful_partition/vmlinuz_hd.vblock"
761 if [ ${FLAGS_skip_vblock} -eq ${FLAGS_FALSE} ]; then
762     if [ $(copy_kernel_vblock) -eq 1 ]; then
763         echo "Copied kernel verification blob from ${VERIFY_BLOB}"
764     else
765         echo "Error: kernel verification blob not found in stateful partition"
766         exit 1
767     fi
768 fi
769
770 POSTINST_BIOS_ARGS=""
771 if [ -n "${FLAGS_target_bios}" ]; then
772     POSTINST_BIOS_ARGS="--bios ${FLAGS_target_bios}"
773 fi
774

```

```
775 # Now run the postinstall script on one new rootfs. Note that even though
776 # we're passing the new destination partition number as an arg, the postinst
777 # script had better not try to access it, for the reasons we just gave.
778 # We can't run this if the target arch isn't the same as the host arch
779 if [ "${FLAGS_skip_postinstall}" -eq "${FLAGS_FALSE}" ]; then
780     loop_offset_setup ${DST} ${START_ROOTFS_A}
781     mount_on_loop_dev
782     IS_INSTALL="1" ${TMPMNT}/postinst \
783         $(make_partition_dev ${DST} ${ROOTFS_A_PART}) \
784         ${SUB_CMD_DEBUG_FLAG} "${POSTINST_BIOS_ARGS}"
785     umount_from_loop_dev
786     loop_offset_cleanup
787 fi
788
789 # Force data to disk before we declare done.
790 sync
791
792 cleanup
793 trap - EXIT
794
795 echo "-----"
796 echo ""
797 echo "Installation to '$DST' complete."
798 echo "Please shutdown, remove the USB device, cross your fingers, and reboot."
```